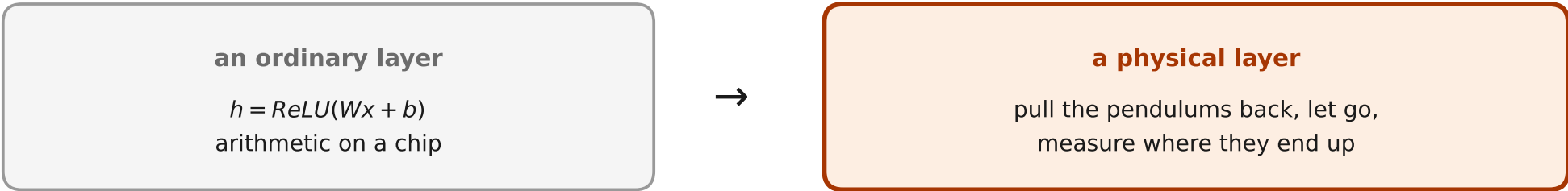


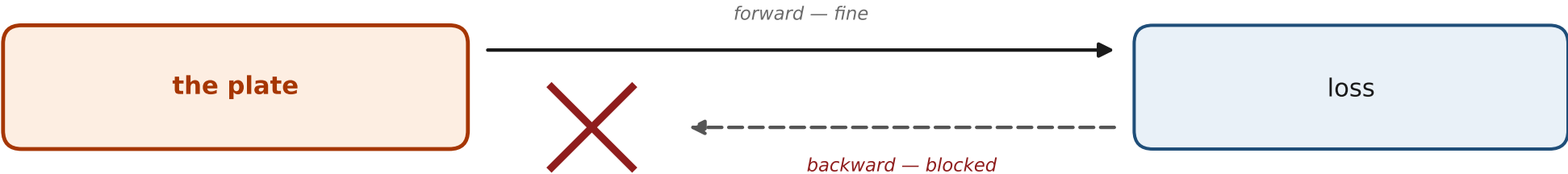
come back here whenever a detail page stops making sense

THE IDEA



same job — mix the inputs, bend them, produce outputs. The weights are the spring stiffnesses.

THE PROBLEM



backprop needs the derivative of every layer. A lump of metal has no code, so it has no derivative.
and you cannot measure your way out: a measurement is a value, not a slope. one poke per knob means millions of runs.

THE FIX



run it, measure it differentiated at the plate's measured values

the simulation is wrong, so the gradient is approximate — but reality re-anchors the state at every layer, so the error never compounds.

THE RESULT

Fashion-MNIST

	tested on the DEVICE what you actually get	tested on the SIMULATOR what you would report
PAT	87.80%	86.55%
trained on the simulation only	79.25%	87.55%

the left column is reality. The right column is what your simulator would tell you — and it says the **WRONG** method won.

The cast

every name used in this deck, defined once — and the one word that causes all the trouble.

TWO MACHINES

THE DEVICE

f_{exp}

hardware — a plate, a crystal, a circuit

- you can RUN it
- you cannot look inside it
- you cannot differentiate it

THE SIMULATOR

f_{model}

code, on your computer

- you can run it AND differentiate it
- it is wrong, by a few percent
- its answers are thrown away — only its slopes are kept

FIVE KINDS OF NUMBER

	what it is	how many numbers	where it lives
x	the data going INTO a layer	a vector	computer → device
θ	the knob settings — these are the weights	millions	the computer, always
$\hat{y}$	what the device MEASURED coming out	a vector	device → computer
L	the loss	ONE number	the computer
g	a gradient — how the loss moves per knob	one per knob	the computer

The word that causes all the trouble

In machine learning, "the model" means the network you are training. The PAT paper calls f_{model} "the differentiable digital model".
Those are two completely different things. This deck never says "the model" on its own:

the network

= what you are training, and it has physical layers

the simulator

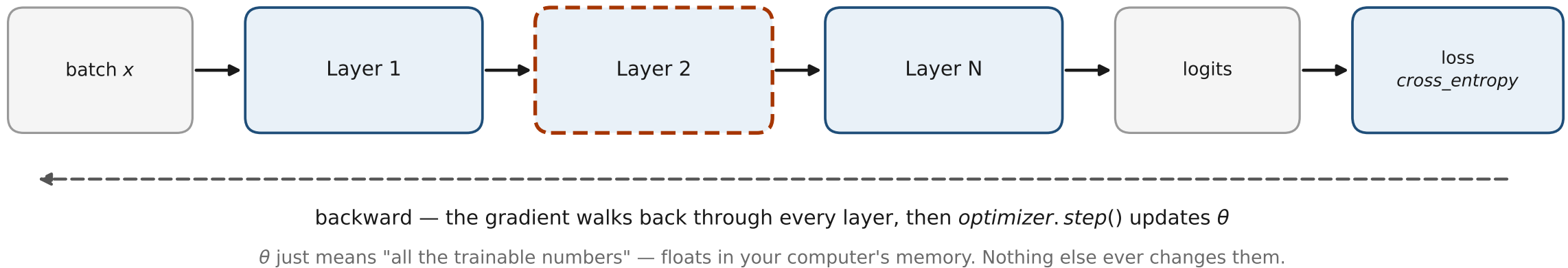
= f_{model}

the device

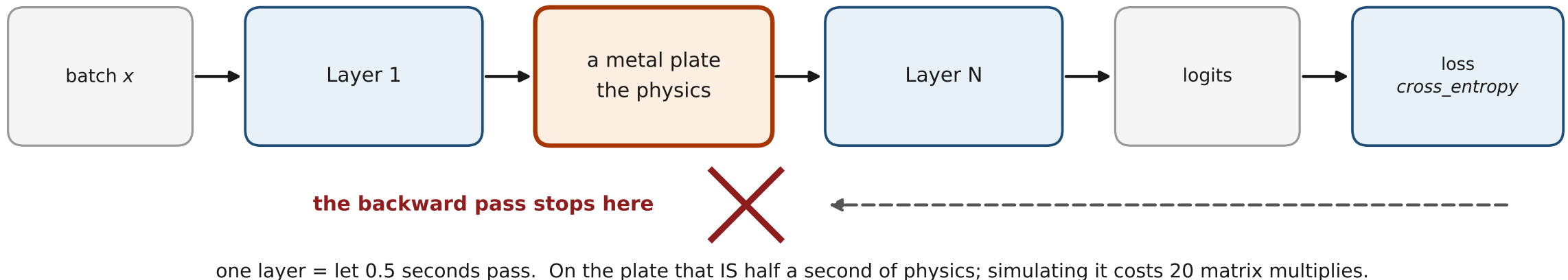
= f_{exp}

a transformer, or any network — the boxes stand for a whole stack. Only one of them changes.

AS YOU KNOW IT



WITH A PHYSICAL LAYER



UNCHANGED

- ✓ the data loading, the other layers, the logits
- ✓ the loss — still $F.cross_entropy(\hat{y}, label)$
- ✓ the optimizer — still Adam
- ✓ θ — still ordinary floats on your computer

BROKEN

- ✗ the plate has no source code
- ✗ autograd recorded nothing on the way through
- ✗ there is no derivative to walk back
- ✗ so this layer cannot be trained

That single broken arrow is the entire problem. Everything that follows is about getting a gradient past it.

"but we ran the real plate and measured the output — why isn't that enough?"

what the device gives you

a VALUE

"with these knobs and this input,
this came out" — one point

what training needs

a SLOPE

"if I nudged knob #4,192 a little,
how would the loss change?"

you cannot read a slope off a single point — a rate of change needs at least two measurements

the same is true of your transformer: knowing the logits tells you nothing about $\partial \text{loss} / \partial W$ until you run backward

You could get the slopes by measurement alone...

measure the loss → nudge one knob → run the plate again → the difference IS that knob's derivative, exactly

then repeat for the next knob. One hardware run per parameter, per gradient step — millions of runs. Honest, and hopeless at scale.

...so why is backprop cheap? Because it knows the STRUCTURE.

it knows every operation that produced the output, so it works out every slope analytically, in a single pass.

measurement knows no structure. It sees a black box, so it must discover each slope separately, one poke at a time.

the device

truth ✓ structure ✗

the simulation

truth ✗ structure ✓

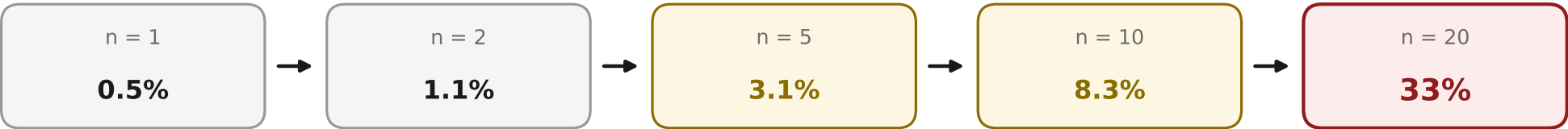
PAT takes from each what it actually has: the device supplies the values, the simulation supplies the slopes.

Fog on a hillside: the altimeter says exactly where you are, the map says which way is down. Read the map at the altimeter's position.

two dead ends, one impossible, and one that works.

	touches hardware during training	can the physics layer learn?	cost per training step	result
A freeze it leave the knobs where they are; train only the digital parts around the plate	no	✗ no	cheap	wastes the device
B simulate both ways train entirely in software, then set the real dials to the finished numbers	no	✓ for the wrong device	cheap	79.25%
C PAT run the real plate; let the simulator differentiate AT the values it measured	yes, every batch	✓ yes	one round trip per layer	87.80%
D measure it directly nudge a knob, re-run the plate, read off the difference — exact, and unaffordable	yes, millions of times	✓ yes	one run per parameter	impossible

Why B loses: suppose the simulation is 0.5% off. After n layers the gap is —



errors compound, they do not add — every layer works on the previous layer's already-wrong answer.

and B never warns you: on its own simulator it reports 87.55%, a quarter-point above PAT's real score of 87.80%. (Fashion-MNIST)

a differentiable stand-in for the device. There are two ways to build one, and neither is exotic.

$f_{\text{model}}(x, \theta) \rightarrow$ a prediction of what the device will do

same inputs as the real device: the data x AND the knob settings θ

it must take θ as an input — otherwise it cannot tell you how turning a knob changes the output

1

Write the physics

you model the mechanism

- write down the equations of motion you believe govern the device
- plug in constants you measured — masses, stiffnesses, damping
- integrate them numerically, in torch

e.g. $\ddot{q} = -\sin q + Q\sin q + e$

your measured constants are off, and the equations were an idealisation to begin with

2

Fit a network to measurements

you imitate the behaviour — no physics written down

- poke the real device thousands of times, sweeping the inputs AND the knob settings
- record every $(x, \theta) \rightarrow y$ pair it produces
- fit a small MLP to them, by ordinary backprop

the simulator IS a neural network: $g(x, \theta) \approx y$

a fit is never exact, and it degrades away from the settings you happened to sample

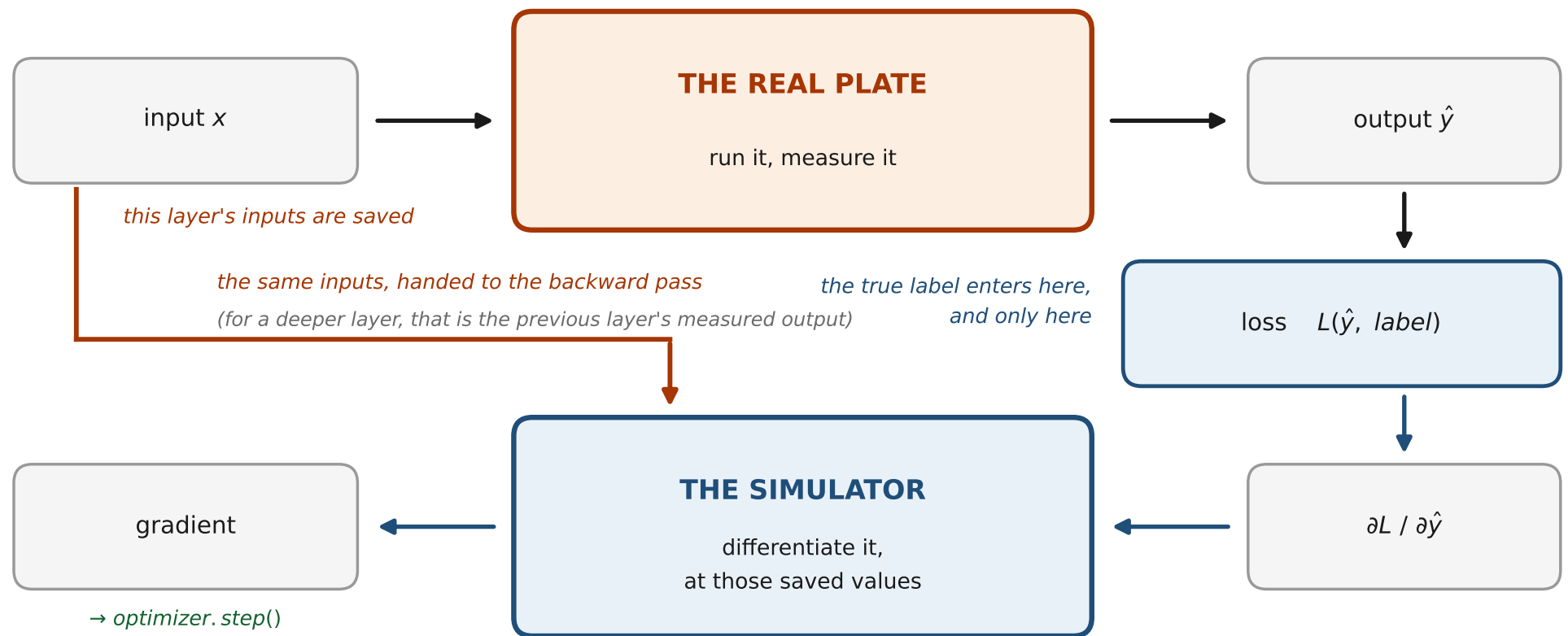
both are wrong, and you can shrink the gap but never close it

What both routes have in common

- built ONCE, offline, before PAT training starts — not learned during it
- it IS executed — you cannot differentiate a function without running it — but its answer is discarded
- the device produces every number the network actually uses; the simulator produces only slopes
- it never has to be good enough to USE. Only good enough to DIFFERENTIATE.

one substitution. Everything else in the training loop is untouched.

FORWARD



The whole method, in about ten lines of PyTorch

```
class func(torch.autograd.Function):
```

```
    def forward(ctx, *args):
```

```
        ctx.save_for_backward(*args)
```

```
        return f_forward(*args)
```

← save this layer's INPUTS — the real ones

← the device runs here, with no graph recorded

```
    def backward(ctx, grad_output):
```

```
        args = ctx.saved_tensors
```

```
        torch.set_grad_enabled(True)
```

```
        y = vjp(f_backward, args, v=grad_output)
```

```
        return y[1]
```

← the inputs saved on the way through

← grad mode switches on only now

← $y[0]$ = the simulator's guess — discarded

← $y[1]$ = the gradient — the only thing used

the gradient is approximate — it comes from a model that is wrong. It only has to point roughly downhill.

two layers. The device is 5% off from the simulator — that is the only difference between them.

the device

$f_{exp}(x, W) = 1.05 \cdot \tanh(Wx)$

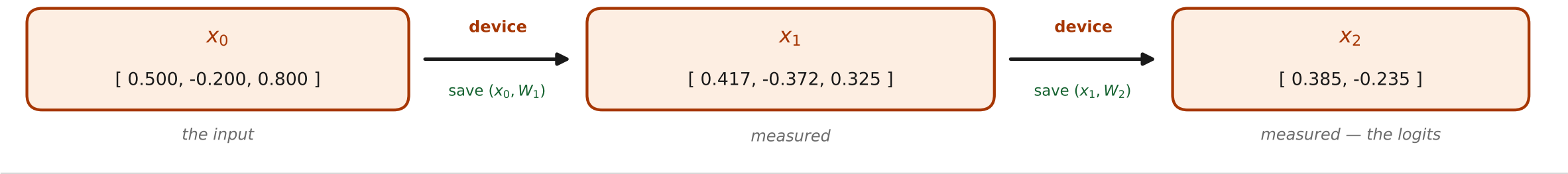
← the 1.05 is the manufacturing error

the simulator

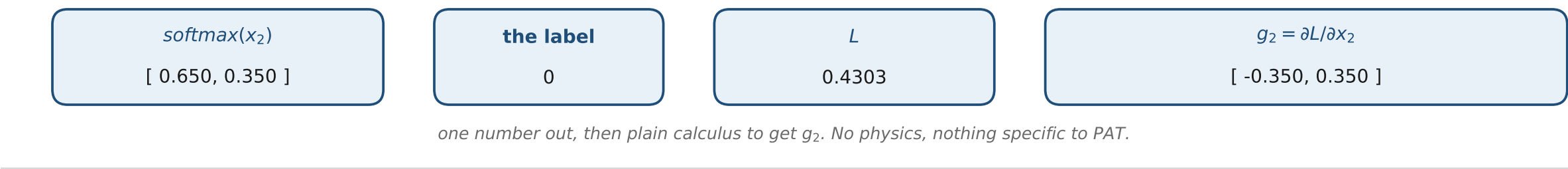
$f_{model}(x, W) = \tanh(Wx)$

← it knows nothing about the 1.05

FORWARD · every number here was MEASURED on the device



THE LOSS · ordinary, on your computer



BACKWARD · differentiate the SIMULATOR, at the values the device saw

layer 2

differentiate f_{model} at (x_1, W_2) , multiply by g_2
the simulator also produced [0.367, -0.224]

gradient for W_2 and $g_1 = [-0.115, 0.163, -0.257]$
← DISCARDED

layer 1

differentiate f_{model} at (x_0, W_1) , multiply by g_1
the simulator also produced [0.397, -0.354, 0.310]

gradient for $W_1 \rightarrow \text{optimizer.step()}$
← DISCARDED

the device measured [0.385, -0.235]. The simulator guessed [0.367, -0.224].
These two are NEVER subtracted, compared, or combined. The right-hand one is deleted.

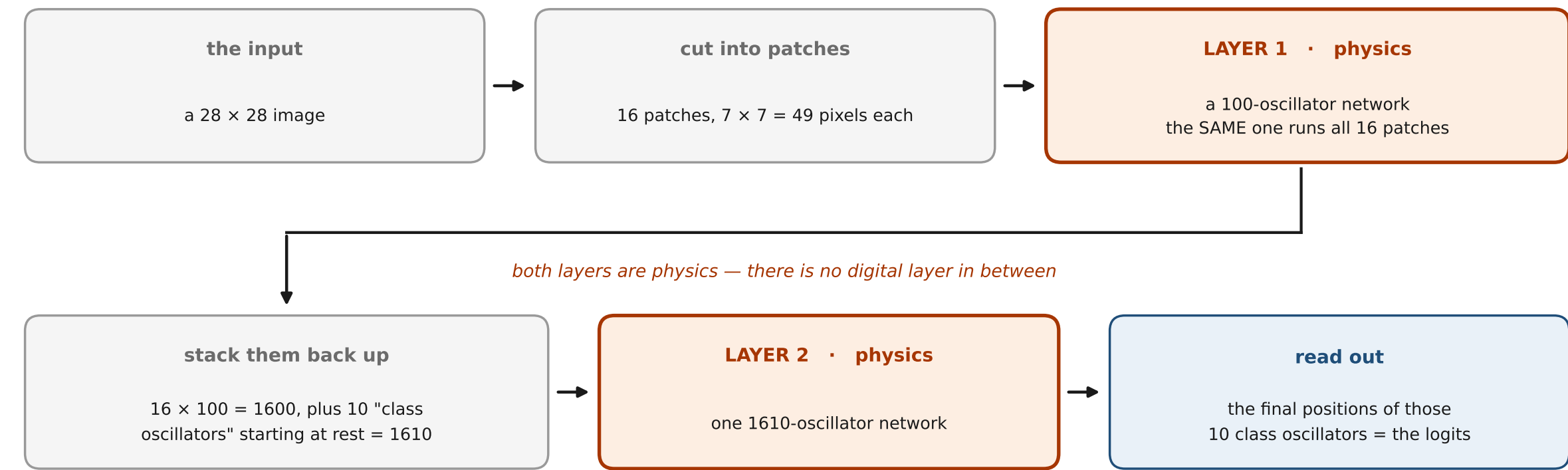
read in file order they look disconnected. Read as 1 → 3 → 2 they are a proof.

	asks	answers	which part of Part One
Example 1 MNIST	can physics be a neural network?	yes — 98%	none of it. This is the CONTROL: no gap, and PAT is not used
forward and backward are the same function, so there is nothing for PAT to fix. It proves the architecture can learn, and deliberately proves nothing about hardware.			
Example 3 a toy scalar function	so can I train in simulation and upload the weights?	no	Step 4 — why option B loses
reality is $2x^{1.1}$, the simulator is $2.01x^{1.1}$: 0.5% off. Apply both 20 times and the gap is 33%. Then the same experiment on gradients: 33% for option B, 10.5% for PAT.			
Example 2 Fashion-MNIST	then how?	PAT	Steps 5 – 7, on a real task
the full method, plus the counterfactual: a second network trained in simulation only, same seed and same budget, then deployed on the device. 87.80% against 79.25%.			

Example 1 read alone looks unimpressive. Read as step one of three, it is doing exactly its job.

Example 3 is the load-bearing one: one number of slack (2.01 against 2.00) and one knob (depth), so every effect is checkable by hand.

two physical layers — not a generic stack — and 2.6 million numbers that never leave your computer.



θ — every trainable number in the network

<code>fc_small.weight</code>	100 × 100	10,000	spring stiffness between every pair, layer 1
<code>fc_small.bias</code>	100	100	the push applied to each oscillator, layer 1
<code>fc_large.weight</code>	1610 × 1610	2,592,100	spring stiffnesses, layer 2
<code>fc_large.bias</code>	1610	1,610	the pushes, layer 2
<code>output_fac</code>	one number	1	a single output scale
TOTAL		2,603,811	ordinary floats on your computer — never inside the plate

They change in exactly one place, and it is the same place as in any torch model: `optimizer.step()`.

both layers are physics. The simulator never appears in the forward pass — it is only ever called on the way back.

The wiring — two lines, and this is the whole mapping

```
f_pat_small = make_pat_func(f_exp_small, f_model)
f_pat_large = make_pat_func(f_exp_large, f_model)
```

1st argument = the FORWARD pass = the device

2nd argument = the BACKWARD pass = the simulator

Where those get called — PNN.forward

```
def forward(self, x):
    x = self.rearrange(x)
    ...
    x = f_pat_small(x, *self.fc_small.parameters())
    ...
    x = f_pat_large(x, *self.fc_large.parameters())
    return self.output_fac * x[:, -10:, 0]
```

← 16 patches of 49 pixels

← LAYER 1 · physics

← LAYER 2 · physics

← the 10 class oscillators = the logits

So, per batch:

	the FORWARD pass runs	the BACKWARD pass runs
layer 1 · 100 oscillators	f_{exp_small} the device	f_{model} at (x_0, C_1, e_1)
layer 2 · 1610 oscillators	f_{exp_large} the device	f_{model} at (x_1, C_2, e_2)
the loss	$F.cross_entropy$	ordinary autograd — nothing special

f_{model} appears zero times in forward(). f_{exp} appears zero times in backward().

Neither is a layer of the network — they are two directions through the same one.